



ENT-164 · Intro to Making

Tufts
UNIVERSITY

Derby Entrepreneurship
Center at Tufts

CLASS 6 · WEEK 6 · FALL 2026

Coding with an AI Agent

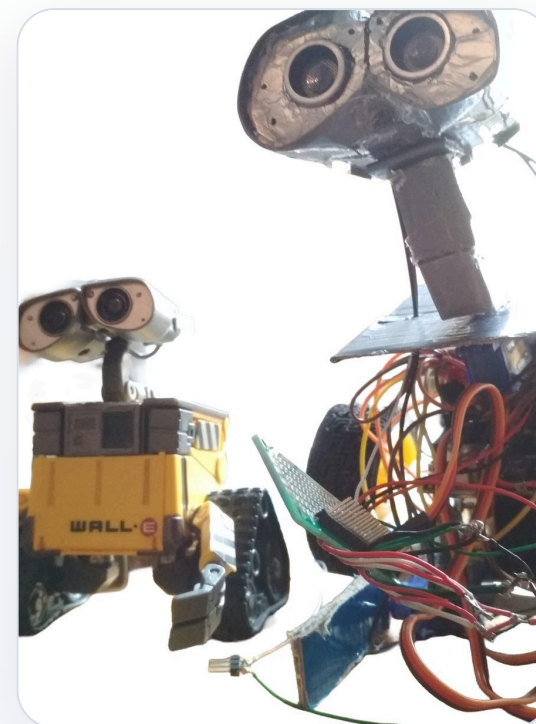
Your ESP32 can serve a web page your phone opens. No app required.
Today we get your robot on WiFi, register it with Tufts, and drive it from
your phone's browser.

Thu, Oct 15

JCC 301 / Nolop

No programming required

Bring your ESP32



Thomas Van de Velde · Instructor · Alexander Beattie · TA · Patrice Pinardo · TA

Tufts University · Gordon Institute

Before we go **wireless**

You should have all three from last class. Tell us now if something is missing.

01

Electronics kit

You've received your electronics kit. The parts you'll wire up today.

02

opencode + MicroPython

opencode is set up and MicroPython is deployed to your ESP32. It runs your code on the board.

03

AI-assisted electronics

You can already build basic electronics projects with the help of AI.

From your phone **to the internet**

Connect, reach out, and build. The three things we do in class today.

01

Connect to your microcontroller

Reach your device from a phone and from a web browser. No native app needed.

02

Connect to external services

Give your device internet access, higher data rates, and cloud services.

03

Drive your robot

Put it all together: a control page on your phone that drives the robot you built.

Two ways to connect

Both work. Only one of them skips app development entirely.

Bluetooth

Short-range, device to device

Low-power, short-range communication. Like connecting to a phone app. Requires a native mobile app for user interaction.

This is hard. You'd have to build and ship an app.

WiFi

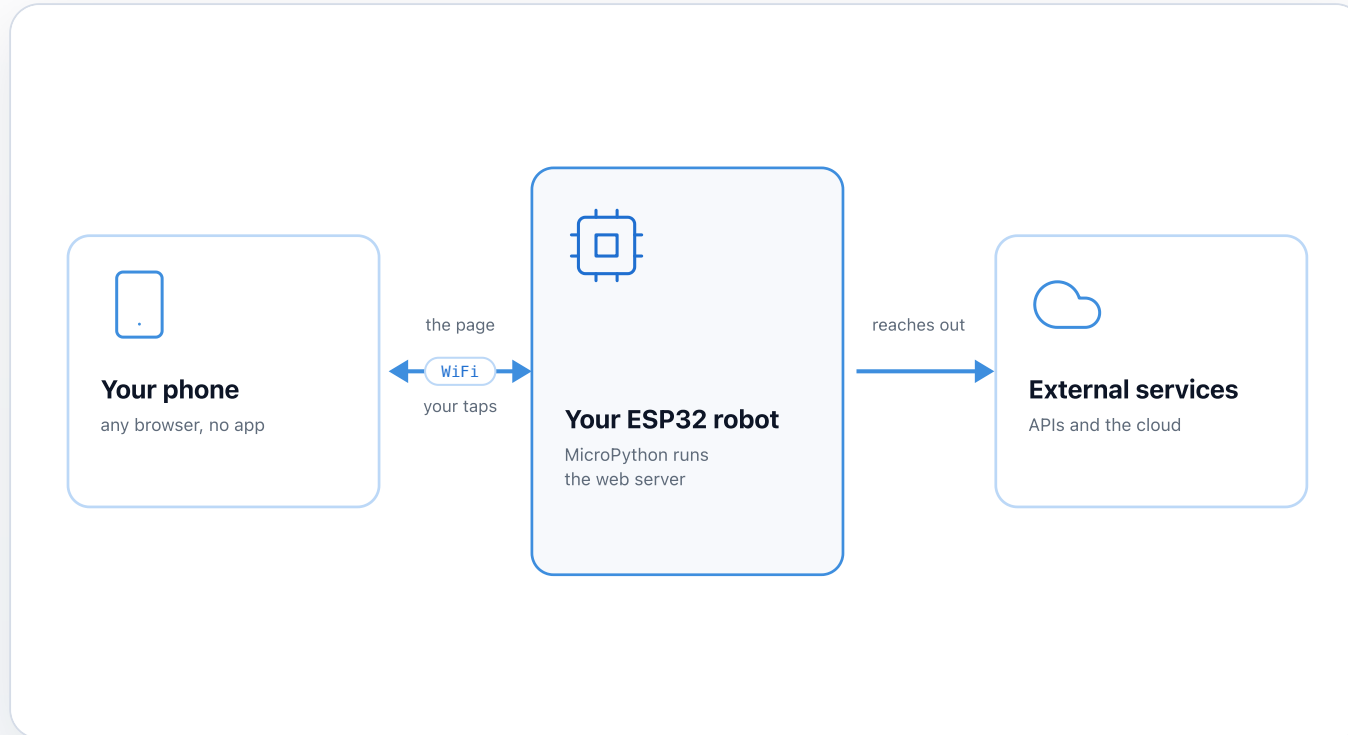
Internet-connected, browser-first

Internet access, higher data rates, and integration with cloud services. Users interact through a simple web browser interface. No app development needed.

Today's approach. The browser is your user interface.

Client, server, or hybrid

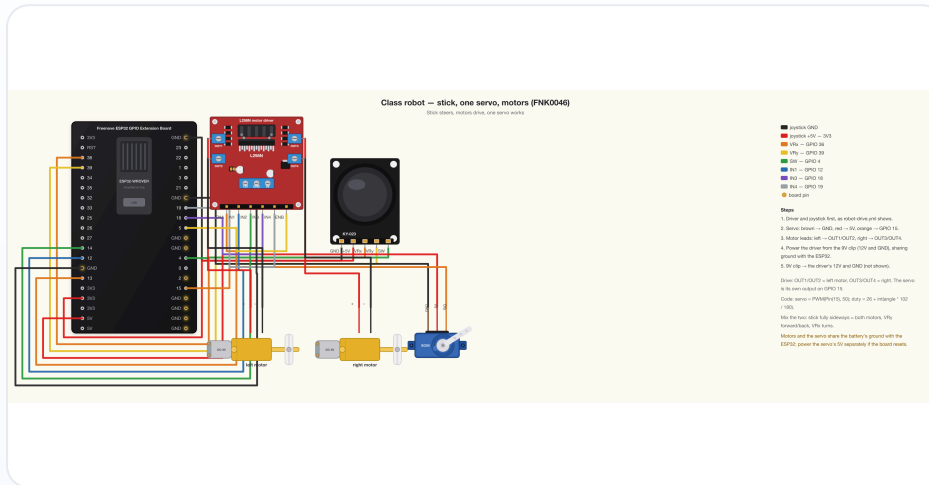
Three pieces: your device, the services it reaches, and the browser you control it from.



Server — the board serves the page · **Client** — the board reaches out to services · **Hybrid** — most projects end up both.

Wiring the drive train

The L298N sits between your ESP32 and the motors — the same wiring your robot already has.



THE DRIVE PINS

ENA → GPIO 13 · ENB → GPIO 5
 IN1 → GPIO 12 · IN2 → GPIO 14
 IN3 → GPIO 18 · IN4 → GPIO 19
 OUT1/2 → left motor · OUT3/4 → right
 12V ← battery pack · shared GND

Wheels **off the ground** while you test. The motors spin the moment the code runs.

Ask for it in plain English

No programming required. Describe the project and let AI write the MicroPython.

THE PROMPT

"I want to use an ESP32 WROVER with MicroPython to drive my robot from a web page on my phone. The motors run through an L298N driver on GPIO 13, 12, 14, 18, 19 and 5. Build a page with forward, back, left, right and stop buttons that run the motors, over the Tufts_Wireless open wifi. Print my ESP32 MAC address on startup."

WHAT THE AI NEEDS FROM YOU

THE BOARD & LANGUAGE

ESP32 WROVER, MicroPython

THE COMPONENT

Two TT motors on an L298N driver

THE GOAL

Drive the robot from the browser

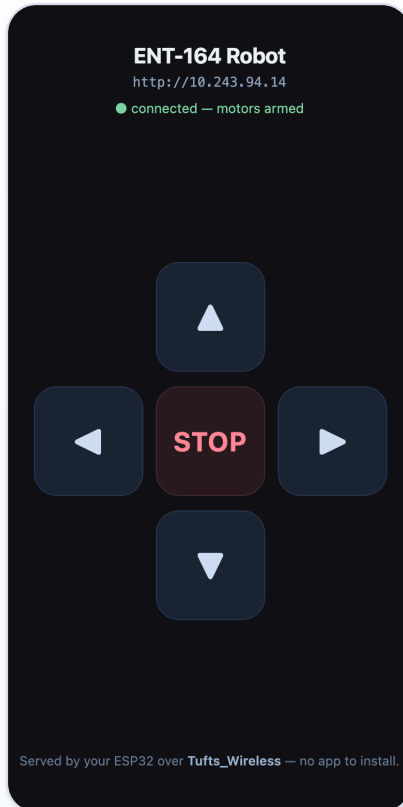
THE CONNECTION

The Tufts_Wireless open WiFi hotspot

The MAC line is extra: it prints the address you'll register with Tufts IT. Next slide.

Every tap is a request

The page is five buttons. Each one hits the ESP32 at a different path.



BUTTON → PATH → WHEELS

- ▲ **Forward** → `/forward` — both motors ahead
- ◀ **Left** → `/left` — left motor back, right ahead
- ▶ **Right** → `/right` — right motor back, left ahead
- ▼ **Back** → `/back` — both motors reverse
- **STOP** → `/stop` — everything off

The board reads the path and sets the driver pins. That is the whole app — the design is the buttons.

Register your ESP32 with Tufts IT

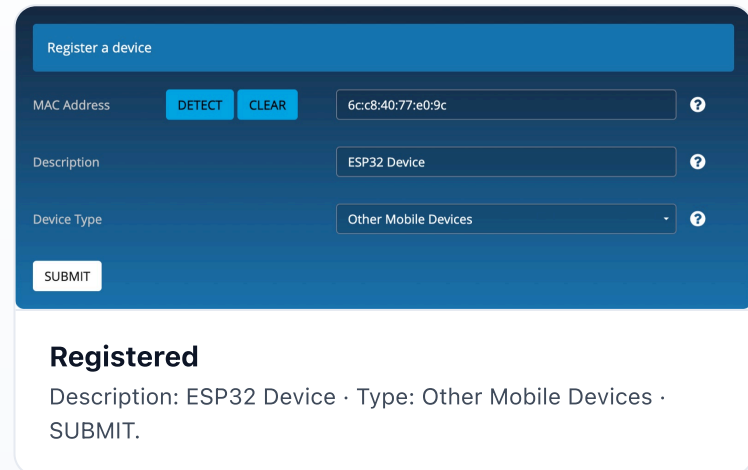
Tufts_Wireless needs your device's MAC address before it will let it join.

01 Run the code with opencode and copy your device's MAC address from what it prints. It looks like `6c:c8:40:77:e0:9c`, but with your own numbers and letters.

02 The rest of the code will fail at this point. You can't reach WiFi yet. That's expected.

03 Go to <https://device-registration.it.tufts.edu>.

04 Clear the default MAC address, then copy your ESP32's MAC from opencode into the field.



The screenshot shows a web form titled "Register a device" with a dark blue header. Below the header, there are three input fields: "MAC Address" with "DETECT" and "CLEAR" buttons, "Description" with a text input, and "Device Type" with a dropdown menu. A "SUBMIT" button is at the bottom. Below the form, a white box displays the status "Registered" and the details "Description: ESP32 Device · Type: Other Mobile Devices · SUBMIT.".

Register a device	
MAC Address	DETECT CLEAR 6cc8:40:77:e0:9c ?
Description	ESP32 Device ?
Device Type	Other Mobile Devices ?
SUBMIT	

Registered
Description: ESP32 Device · Type: Other Mobile Devices · SUBMIT.

Use the IP address, **not the MAC**

opencode prints two addresses. Only one of them belongs in your browser.

Use this

192.168.6.134

Type the IP address opencode prints into your phone's browser, e.g. `http://192.168.6.134`. That opens your device's page.

Not this

6c:c8:40:77:e0:9c

The MAC address was only needed for Tufts IT registration. It will not open a web page.

ATTENTION It may take a few minutes before your device registration is active. If the page doesn't load, wait and refresh.

What happened?

Every tap on your phone becomes a request the ESP32 answers.

A web server starts

1

Your ESP32 runs a web server that renders a web page. No separate computer needed.

The page gets an address

2

The page is accessible at the device's IP address, assigned automatically when it joins Tufts_Wireless.

Your phone renders it

3

Your phone's browser renders the HTML that the ESP32's web server generates.

Requests round-trip

4

Tapping a button sends a request to the ESP32 (e.g. `http://10.243.94.14/forward`). It reads the path, sets the motor pins, and answers with the next page state.

Other things to try

Once your device is online, the browser becomes a remote control for anything you build.

01

Visualize sensor data

A dashboard with temperature history charts.

02

Joystick driving

Swap the buttons for a virtual joystick: drag to steer, release to stop.

03

Manage settings on the phone

Save them to the device: set a thermostat temperature, or set an alarm clock.

04

Device state visualization

See the status of an LED (on/off) or the current servo angle.

BEFORE YOU GO

Questions?

Then get your device online. Next week: Final Robot Assembly and Testing. Integrate everything you've learned into a working robot.

Individual assignment: electronics project. Present a working circuit.

WHERE TO FIND US

Office hours

`go.tufts.edu/meet_tvdv`

Email

`thomas.van_de_velde@tufts.edu`

Course materials

`tuftsmaker.github.io/ENT-164`